CODENTIA

AI Policy Layer

Complete Project Guide

How to Run • How It Works • Technologies • Architecture • Workflow • Demonstration

Project: Codentia
Repository identity: santoshkandasani/Codentia
Core concept: Protect AI interactions before the prompt reaches the AI and validate the AI response before it reaches the user.

This guide is written for project demonstration, development, viva/judging, and setup.

1. Project Overview

Codentia is a browser-level AI policy layer. Instead of allowing an AI interaction to move directly from the user to an AI platform, Codentia introduces a policy checkpoint before the request and another checkpoint after the AI response.

The central idea is a two-sided security and governance flow:

```
USER → CODENTIA PRE-CHECK → AI → CODENTIA POST-CHECK → USER
```

The pre-check can identify supported sensitive patterns such as email addresses, phone numbers, card-like/long numeric identifiers, PAN-like identifiers, Aadhaar-like identifiers, API/access-key patterns, and password/secret/token patterns. Depending on the policy, information can be masked or a high-risk request can be blocked.

The post-check examines the generated output for supported sensitive patterns and can flag output that should be redacted, warned about, or blocked by the surrounding policy logic.

> **Important:** Pattern detection is not the same as perfect semantic understanding. A simple pattern detector can produce false positives or miss information that has an unusual format. For a production system, stronger entity recognition, contextual classification, and platform-specific integrations would be needed.

2. What Problem Does Codentia Solve?

Employees, students, developers, and organizations increasingly use external AI services. The problem is not only what the AI generates; it is also what users accidentally send to it.

Problem	Codentia response
User pastes personal information into an AI prompt	Pre-check detects supported PII patterns and can mask them.
Developer pastes an API key or secret	Secret-like patterns can trigger blocking.
Organization needs a common AI-use policy	Rules can be applied as a policy layer instead of relying only on user memory.
AI output contains sensitive information	Post-check can detect supported patterns and flag/redact them.
Different AI platforms are used	The browser-extension approach provides a common enforcement point, subject to each platform's UI and techni

3. Core Workflow

```
1. User enters a prompt
↓
2. Codentia intercepts/inspects the interaction
↓
3. Pre-check scans for configured sensitive patterns
↓
4. If detected → mask / warn / block according to policy
↓
5. Protected prompt continues to the AI
↓
6. AI generates a response
↓
7. Codentia post-check scans the response
↓
8. Allow / warn / redact / block according to policy
↓
9. User receives the permitted response
↓
10. Event can be recorded in local activity/audit state
```

4. End-to-End Example

Policy: “Do not send customer PII to an external AI.”

User enters:

```
Customer: Rahul Kumar
Email: rahul@gmail.com
Phone: 9876543210

Summarize this customer's complaint.
```

Codentia pre-check:

```
Email address → detected
Phone number → detected
Action → protect sensitive values
```

Protected prompt:

```
Customer: Rahul Kumar
Email: [EMAIL REDACTED]
Phone: [PHONE REDACTED]

Summarize this customer's complaint.
```

The AI can now perform the summarization without receiving the original email and phone number. Codentia then examines the AI response for supported sensitive patterns before the response is shown as compliant.

5. Technologies Used — Detailed

Technology	Where used	Why it is used
HTML5	Demo page, popup, dashboard/UI pages	Provides the structure for user interfaces and the standalone demonstratio
CSS3	Extension popup, demo page, dashboard styling	Controls layout, spacing, typography, states, warnings, and responsive pre
JavaScript	Chrome extension logic and demo logic	Implements prompt/response interception logic, pattern detection, masking
Chrome Extension APIs	Extension background/content/popup components	Connects Codentia to browser pages and allows the extension to observe
Regular Expressions (RegExp)	PII/secret detection	Detects structured patterns such as emails, phone numbers, card-like num
MOSS / WASM-related components	Existing project extension assets	The original project includes code-similarity related components. They are
Next.js / React ecosystem	Dashboard/project web UI where present	Provides a modern component-based web application structure for the das
Node.js + npm	Dashboard development/build	Runs JavaScript tooling and installs project dependencies.
Local browser storage/state	Policy/activity configuration	Keeps browser-side settings and activity information without requiring a re
Git/GitHub	Source control and project distribution	Project identity can be maintained under santoshkandasani/Codentia.

6. Technology Flow — Where Each Part Fits

7. Detailed Component Responsibilities

Content script: Runs in supported web pages and interacts with the page DOM. It is the component that can inspect the visible AI-chat interface and identify inputs/responses according to the selectors implemented by the project.

Background/service logic: Handles extension-level coordination, browser messaging, state, and tasks that should not be tied directly to a single webpage.

Popup/UI: Gives the user a place to view or configure policies and extension behavior.

Detection engine: Uses deterministic pattern checks for supported data formats. For example, an email is recognized through an email-shaped regular expression.

Masking engine: Replaces detected sensitive values with safe placeholders such as `[EMAIL REDACTED]`.

Post-check: Applies the same or related detection logic to the AI-generated response.

Demo page: Provides a controlled environment for explaining the architecture without relying on the DOM structure of an external AI site.

8. How to Run Codentia V2

Recommended order: first run the standalone demo, then load the Chrome extension, and only after that test integration with a supported AI website.

A. Extract the project

Extract `Codentia_V2.zip` to a convenient folder, for example:

```
C:\Users\santo\Downloads\Codentia_V2
```

B. Run the standalone demonstration

No Python server is required for the standalone HTML demonstration.

```
1. Open the extracted Codentia_V2 folder.
2. Find codentia-demo.html.
3. Double-click the file.
4. It opens in Chrome.
5. Click “Run Codentia Pre-check”.
6. Observe the detected fields and protected prompt.
7. Review the simulated AI response.
8. Click “Run Codentia Post-check”.
```

C. Load the Chrome extension

```
1. Open Google Chrome.
2. Navigate to:
chrome://extensions
3. Turn on Developer mode.
4. Click “Load unpacked”.
5. Select:
Codentia_V2\extension
6. Confirm that the Codentia extension appears.
```

D. Test the extension

Open a supported AI website and test a harmless rule first. For example, configure a rule such as “Keep answers under 50 words” or “Always use bullet points.” Then test the pre-check using a sample email/phone number that is not real personal data.

E. Dashboard, if included in the project package

If the dashboard directory contains a `package.json`, open PowerShell in that directory and run:

```
node -v
npm -v
npm install
npm run dev
```

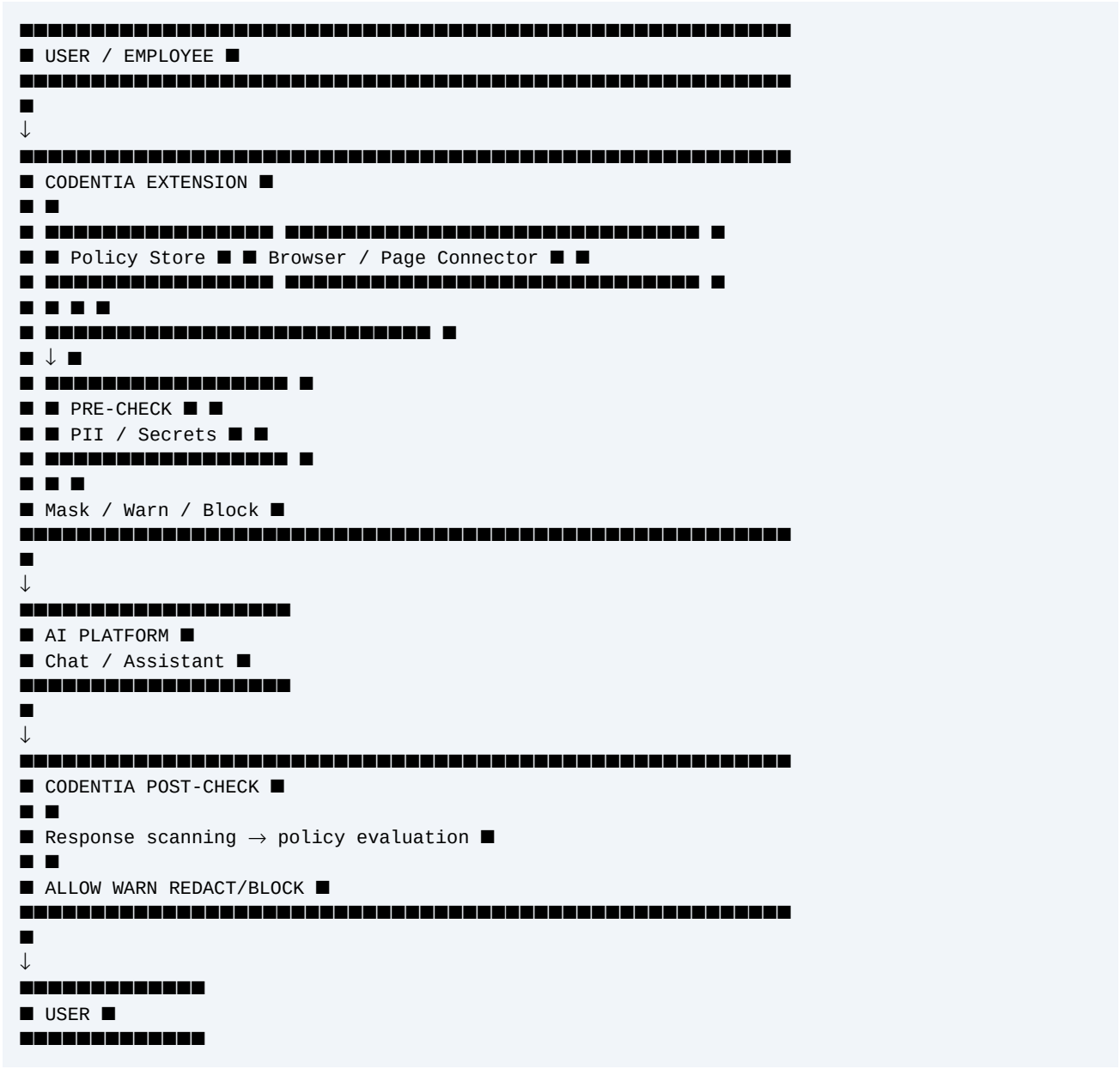
Then open the localhost URL printed by the development server, commonly `http://localhost:3000`. The exact port is determined by the project's Next.js configuration.

9. Troubleshooting

Problem	Check
Chrome says the extension cannot load	Read the exact error on chrome://extensions. Common causes are manifest errors, missing referenced
Extension loads but does nothing on an AI site	The site's DOM/selectors may not match the extension's current selectors. Inspect content-script permis
Dashboard npm command fails	Run node -v and npm -v, then run npm install inside the directory containing package.json.

Problem	Check
PII is not detected	The detector is pattern-based. Check whether the test value matches one of the supported formats.
A normal number is detected as sensitive	Pattern detectors can create false positives. Use contextual rules or improve the detector before claimin
The AI output is not checked	Verify that the response observer/content script is active on the target page and that the site is supporte

10. System Architecture



11. Policy Decision Model

Result	Meaning	Example
ALLOW	No relevant policy violation detected.	Normal question with no sensitive pattern.
WARN	Potential violation detected but policy permits user review.	Non-critical information appears in a response.
MASK / REDACT	Sensitive value is replaced before continuing or displaying.	Email becomes [EMAIL REDACTED].
BLOCK	The interaction should not continue under the configured policy.	A prompt contains an API secret under a strict security policy.

12. What Codentia Checks in the Current V2 Demo

• **Email:** email-shaped strings.

• **Phone:** common Indian mobile-number pattern and +91 variant.

• **Card/long numeric identifiers:** long digit sequences matching the detector.

• **PAN-like identifier:** five letters + four digits + one letter pattern.

• **Aadhaar-like identifier:** 12-digit grouped pattern.

• **Password/secret/token assignments:** labels such as API key, secret, password, passwd, or access token followed by a value.

• **API/access-key patterns:** selected key-like prefixes such as sk- or AKIA-like patterns.

These are deterministic pattern checks. They are useful for a prototype and demonstration, but they are not a complete PII/secret classification system.

13. Recommended Live Demonstration

```
DEMO 1 — Privacy protection
1. Enter a customer complaint containing a sample email and phone.
2. Run PRE-CHECK.
3. Show the detected categories.
4. Show the protected prompt.
5. Explain that the original values should not be sent to the AI.

DEMO 2 — Secret protection
1. Load the API-key example.
2. Run PRE-CHECK.
3. Show the security warning/block behavior.

DEMO 3 — Output protection
1. Put a sample sensitive value into the simulated AI response.
2. Run POST-CHECK.
3. Show that the output is flagged.

DEMO 4 — Normal interaction
1. Load the clean example.
2. Run PRE-CHECK.
3. Show that the request passes normally.
```

14. What to Say to a Judge/Evaluator

Short explanation: “Codentia is an AI policy layer. It sits between the user and an AI platform. Before the request reaches the AI, Codentia checks for sensitive information and applies a policy such as mask, warn, or block. After the AI responds, Codentia checks the output again. This gives organizations a common enforcement point for responsible AI usage.”

Technical explanation: “The browser extension uses content scripts and browser messaging to interact with supported AI web pages. A deterministic detection layer uses regular expressions for structured sensitive-data patterns. A masking layer replaces detected values with safe placeholders. The response path is checked again after generation. Policies and extension state can be maintained on the client side for the basic prototype.”

Important limitation to state honestly: “The current detector is pattern-based and the browser integration depends on the target website's DOM and permissions. It is a prototype policy layer, not a universal or production-grade PII classifier.”

15. Strong Future Improvements

Improvement	Purpose
Context-aware NER	Detect real person names, organizations, locations, and other entities instead of relying only on patterns.
Platform adapters	Create separate adapters for ChatGPT, Claude, Gemini, Perplexity, etc., because their DOM structures diffe
Policy packs	Education, enterprise privacy, developer security, and personal privacy policies can be enabled as reusable
Central admin console	Organizations can define policies centrally and distribute them to users.
Audit dashboard	Show allowed, warned, masked, and blocked events over time.
Policy simulator	Test a rule against a prompt/response before deploying it.
Secret scanning	Expand support for cloud credentials, JWTs, private keys, database URLs, and other credential formats.
Local-only processing	Keep sensitive detection local in the browser wherever possible so the detector itself does not transmit privat
Tamper-resistant policy delivery	For enterprise deployments, prevent ordinary users from disabling mandatory security policies.
Automated testing	Build test suites for each supported AI website and every detector pattern.

16. Final Project Workflow at a Glance

17. Final Takeaway

Codentia's core value is not simply adding another instruction to an AI prompt. The stronger architecture is a **policy checkpoint around the AI interaction**: inspect what leaves the user, protect sensitive data before it reaches the AI, inspect what comes back, and enforce the configured policy before the result reaches the user.

For the current project, the standalone demo is the safest way to demonstrate the complete concept. For a stronger production implementation, the next engineering priorities should be contextual PII detection, platform-specific adapters, robust policy storage, and comprehensive automated tests.